**Task 1:** Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

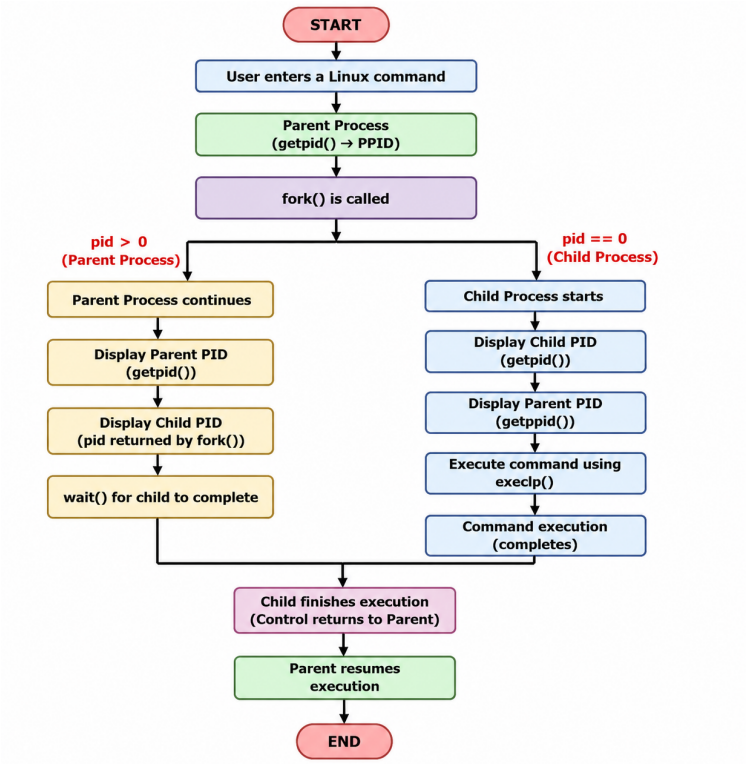
a) **Write a C code to create a process using fork() system call.**

b) **Read the user input as shell command.**

c) **Use a system call exec() to execute the shell command.**

d) **Use a wait() system call for waiting a parent process.**

e) **Display the PIDs of parent and child processes.**




**Example Output:**

Enter a Linux command: ls

Parent Process

Child Process

Parent PID : 67

Child PID : 68

Child PID  : 68

Parent PID : 67

Executing command: ls

CE1  FIRST.c  File2.c  Proces_States.c  command_executor  filecopy.c
hello.c  process_example  session_4_a  session_4_b.c

CE1.c  File1.c  First.c  Process_States.c  command_executor.c  fork_example.c.save
my_shell  process_example.c  session_4_a.c

FIRST  File1.txt  Proces_States  R1  filecopy  hello
os-lab  process_example.cyes  session_4_b

Child process completed.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[256];
    pid_t pid;

    // a) & b) Read the user input as a shell command
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';   // strip trailing newline

    // a) Create a process using fork()
    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        exit(1);
    }
    else if (pid == 0) {
```

```
        // ---- Child Process ----
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command: %s\n", command);

        // c) Execute the command using exec()
        execlp("/bin/sh", "sh", "-c", command, NULL);

        // Only reached if execlp() fails
        printf("Command execution failed.\n");
        exit(1);
    }
    else {
        // ---- Parent Process ----
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        // d) Wait for the child process to finish
        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

**Output:**

```
Enter a Linux command: ls
Parent Process
Parent PID : 67
Child PID : 68
Child Process
Child PID : 68
Parent PID : 67
Executing command: ls
CE1 FIRST.c File2.c Proces_States.c command_executor filecopy.c hello.c
process_example session_4_a session_4_b.c
CE1.c File1.c First.c Process_States.c command_executor.c fork_example.c.save
my_shell process_example.c session_4_a.c
FIRST File1.txt Proces_States R1 filecopy hello os-lab process_example.cyes
session_4_b
Child process completed.
```

**Task-2:** Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

**`Report`***:*

I started with `uname-a` to see the kernel name, version, and CPU architecture. The kernel is the only piece of software that talks to the hardware directly — everything else, including my own programs, goes through it.

Then I ran `lscpu` to check the actual CPU — how many cores, threads, and clock speed it has. Even with just a few physical cores, dozens of programs can seem to run at once. That's not really true at the hardware level — the OS scheduler just switches the CPU between processes so fast it looks simultaneous. That's multitasking.

Next, `ps -ef` listed every running process with its PID and parent PID. This is the OS's process abstraction in action: each program looks like it has its own private memory, even though every process is really sharing the same physical RAM. The OS handles this through virtual memory — mapping each process's private-looking address space to real physical memory using page tables, which also keeps processes from stepping on each other.

`top -n 1 -b` gave a live snapshot of CPU and memory usage across processes. This shows the scheduler constantly reallocating CPU time and memory based on its scheduling policy (Linux uses CFS by default), rather than handing resources out once and leaving them alone.

Storage and I/O work similarly. I never touch raw disk blocks — the filesystem translates files and folders into physical locations for me. I/O devices work the same way: the OS gives a uniform interface (often treating devices like files under `/dev/`), so a program can just call `read()` or `write()` without knowing anything about the actual hardware.

Overall, this shows the OS's real job: take limited physical hardware — CPU, RAM, disk, I/O — and make it look like every program has its own dedicated, simple version of it. That illusion is what makes multitasking, process isolation, and running the same software on different machines possible.